

PYTHON: ERRORI TIPICI

Scheda di ripasso — errori ricorrenti nelle prime lezioni

NOMI DA NON USARE

- list, str, int, len, type, sum, print sono nomi già usati da Python
- Se li sovrascrivi come variabile, perdi la funzione originale nello script

Esempio 1 — errore da evitare:

```
list = phrase.split("-")
# ora list(...) non funziona più
# come funzione, per il resto del codice
```

Esempio 2 — nomi sicuri da usare:

```
pezzi = phrase.split("-")
lista_numeri = [1, 2, 3]
testo = "ciao" # ok, non è riservato
```

SPLIT: OCCHIO AGLI SPAZI

- split(sep) taglia ESATTAMENTE sul separatore, spazi inclusi nei pezzi
- Meglio non fidarsi 'a occhio': controllare sempre il risultato con print()

Esempio 1 — spazi che restano nei pezzi:

```
f = "A - B - C"
print(f.split("-"))
# ['A ', ' B ', ' C'] (spazi inclusi!)
```

Esempio 2 — join: metti tu il separatore giusto:

```
pezzi = ["A", "B", "C"]
print(" / ".join(pezzi))
# A / B / C (spazi voluti, non casuali)
```

METODI CONCATENATI E IMMUTABILITÀ

- Le stringhe sono immutabili: un metodo restituisce SEMPRE una stringa nuova
- Si possono incatenare più metodi in una riga, eseguiti da sinistra a destra

Esempio 1 — non modifica l'originale:

```
t = "ciao"
t.upper()
print(t) # ciao (invariata!)
t = t.upper() # così sì
```

Esempio 2 — incatenare più metodi:

```
s = "  mario rossi  "
print(s.strip().title())
# Mario Rossi
```

ALTRI ERRORI FREQUENTI

- = assegna, == confronta; input() restituisce sempre testo (serve int())
- return dentro un for esce alla PRIMA iterazione: va messo dopo il ciclo

Esempio 1 — return nel posto sbagliato:

```
def conta_pari(lista):
    for n in lista:
        if n % 2 == 0:
            return n # NO: si ferma subito
```

Esempio 2 — corretto, return dopo il ciclo:

```
def conta_pari(lista):
    c = 0
    for n in lista:
        if n % 2 == 0:
            c += 1
    return c # SÌ: dopo il for
```